SYSTEM DESIGN ENGINEERING SYLLABUS

MODULE 1: JAVA & OBJECT ORIENTED PROGRAMMING FOUNDATIONS

You will build a working Library Management System by the end of this module

Core Java

- JVM, JDK and JRE – how Java actually runs
- Primitive vs reference types – stack vs heap memory model
- Classes, objects, constructors, this keyword
- Access modifiers – visibility as a design decision

OOP Pillars

- Encapsulation – hiding state, exposing behaviour
- Abstraction – interfaces vs abstract classes, when to use each
- Inheritance – what it gives you and when it is overused
- Polymorphism – runtime vs compile time, eliminating instanceof
- Method overloading vs overriding
- Composition vs inheritance – HAS-A vs IS-A
- OOP best practices and common mistakes

Case Study: Library Management System

- Issue book, return book, search, fine calculation
- Deliberate OOP violations seeded in starter code – students identify and fix them

MODULE 2: SOLID PRINCIPLES & CLEAN CODE

You will refactor two real legacy systems applying all five principles

Design Smells – recognising bad code

- God class, feature envy, primitive obsession, anemic domain model

SOLID Principles

- Single Responsibility – one reason to change, not one method
- Open/Closed – extend without modifying existing code
- Liskov Substitution – the Square/Rectangle problem and why it matters
- Interface Segregation – fat interfaces and how to split them
- Dependency Inversion – depend on abstractions, not concretions
- Dependency injection – the idea, not the framework

Case Studies

- Invoice Management System – apply SRP, OCP, DIP
- Order Management System – full SOLID refactor of legacy code
- Before vs after comparison – measuring what each principle fixed

MODULE 3: UML & OBJECT MODELING

Just enough UML to communicate design clearly in interviews

- Class diagrams – the only diagram that matters in LLD interviews
- Sequence diagrams – how objects talk to each other over time
- ER modeling – translating domain models to data models
- Domain modeling – finding entities, relationships, and boundaries
- Hands-on: produce class and sequence diagrams for the Library System from Module 1

MODULE 4: DESIGN PATTERNS

Every pattern is taught through a project – never pattern first, project later

Creational Patterns

- Singleton – thread-safe implementation, when it becomes a smell
- Factory & Abstract Factory – eliminating new ConcreteClass() everywhere
- Builder – solving the telescoping constructor problem
- Project: Payment Gateway Factory – UPI, Card, NetBanking created via factory

Structural Patterns

- Adapter – making incompatible interfaces work together
- Decorator – adding behaviour without subclassing, Java I/O as a real example
- Proxy – access control, lazy loading, caching
- Project: Vending Machine – Singleton + Factory + State preview

Behavioural Patterns

- Strategy – replacing if-else chains on behaviour, pluggable algorithms
- Observer – one change, many reactions – stock prices, order status updates
- Command – encapsulating requests as objects, undo/redo support
- Template Method – fixed skeleton, overridable steps

- State – replacing status field conditionals with explicit state objects
- Chain of Responsibility – middleware pipelines, approval workflows
- Project: Smart Home System – Command with undo, Observer for device events
- Project: Loan Approval Workflow – Chain of Responsibility + Strategy

MODULE 5 : JAVA CONCURRENCY & MULTITHREADING

You will build systems where multiple threads interact safely

Foundations

- Threads vs processes – what actually runs concurrently
- Runnable vs Callable vs Thread – when to use each
- synchronized keyword – what it protects and what it misses
- volatile – visibility guarantee, not atomicity
- wait(), notify(), notifyAll() – producer-consumer model

Executor Framework

- Why raw thread creation is dangerous at scale
- ExecutorService, ThreadPoolExecutor – managing worker pools
- Future and Callable – getting results from async tasks
- CountDownLatch, CyclicBarrier, Semaphore – coordination primitives

Case Studies

- Printer Queue System – multiple threads, one processor, no corruption
- Order Fulfillment System – payment, inventory, notification running concurrently via ExecutorService

MODULE 6: ADVANCED CONCURRENCY

You will build production-grade thread-safe systems

Concurrent Data Structures

- ConcurrentHashMap – why it is not just synchronizedMap
- BlockingQueue – clean producer-consumer without manual locking
- CopyOnWriteArrayList – when reads dominate
- AtomicInteger, AtomicReference – lock-free operations via CAS

Advanced Locking

- ReentrantLock vs synchronized – tryLock, timed lock, explicit control
- ReadWriteLock – concurrent readers, exclusive writer

- Deadlock – four conditions, how to design around it
- Livelock and starvation – less discussed, equally real
- CompletableFuture – chaining async operations

Case Studies

- Thread-Safe LRU Cache – LinkedHashMap + ReadWriteLock + TTL expiry
- Banking Transfer System – concurrent transfers without deadlock
- Log Aggregator – concurrent producers, batching consumer, disk writes

MODULE 7: LOW LEVEL DESIGN CASE STUDIES

Full LLD interviews – entities, patterns, thread safety, and code

- Each case study covers: requirement gathering, class design, pattern identification, concurrency handling, and how to explain it in an interview.
- Parking Lot – ConcurrentHashMap + spot locking, Strategy for fee calculation
- Elevator System – State pattern, SCAN scheduling strategy, concurrent request queue
- ATM Machine – State + Command with undo, atomic balance updates
- Movie Ticket Booking – seat locking under concurrency, waitlist via Observer
- Splitwise – Strategy for split types, balance simplification, thread-safe group updates
- Snake and Ladder – Factory for cell types, Strategy for dice, State for game lifecycle
- Cab Booking System – AtomicReference for driver state, State for trip lifecycle
- Food Ordering System – Builder for orders, State for order tracking, Strategy for discounts
- Hotel Booking System – date overlap detection, Strategy for pricing and cancellation
- Advanced Library Management System – revisit Module 1 with all patterns and concurrency applied
- Logging Framework – Singleton registry, Chain of Responsibility for filtering, async via BlockingQueue
- Notification System – Strategy for channels, Builder for messages, ExecutorService for bulk dispatch
- In-Memory Key-Value Store – ConcurrentHashMap + TTL expiry + Observer for pub/sub
- Rate Limiter – token bucket, sliding window, AtomicInteger for thread-safe counting

MODULE 8: SYSTEM DESIGN FUNDAMENTALS

How to approach any HLD problem in 45 minutes

The Framework

- Requirements → estimation → components → deep dives → tradeoffs
- Functional vs non-functional requirements – what to ask before drawing
- How to scope and timebound a design under interview pressure

Scalability and Availability

- What scalability means at 1K, 1M, and 1B users – decisions change at each level
- Availability targets – 99.9% vs 99.99% – what that means in downtime per year
- Reliability vs availability – why they are not the same
- Real example: how WhatsApp scaled to 1 billion users with 50 engineers

Capacity Estimation

- Storage, bandwidth, request volume from first principles
- Peak vs average traffic, read/write ratio, 5-year data growth
- Worked example: estimate Instagram – photos per day, CDN cost, DB size

Networking

- DNS, HTTP, HTTPS – where they appear in system design diagrams
- WebSockets vs SSE vs polling – Uber uses WebSockets, Gmail uses SSE – why different
- Reverse proxy and load balancers – L4 vs L7, placement decisions

Practical: API Exploration

- Hit real APIs via Postman – observe pagination headers, rate limit responses, idempotency keys in the wild
- What a 429, 503, 504 tells you about the system behind it

Case Study

- Instagram Architecture

MODULE 9: API DESIGN & AUTHENTICATION

Real APIs used as examples throughout

API Design

- REST principles – resources, verbs, status codes as design decisions
- Pagination – offset vs cursor vs keyset – why Twitter moved away from offset
- Idempotency – how Stripe prevents double charges using idempotency keys
- API versioning – path vs header vs query param tradeoffs

Authentication & Authorization

- AuthN vs AuthZ – the conceptual difference
- JWT, OAuth 2.0, API keys – when to use each at architecture level
- Session vs token based auth – stateful vs stateless tradeoffs
- How Google OAuth works at a system design level

Infrastructure

- API Gateway – routing, auth, rate limiting, aggregation in one place
- BFF pattern – why Netflix serves different response shapes to TV vs mobile vs web

Case Studies

- GitHub API – pagination, rate limiting, token auth in practice
- Payment APIs – idempotency, webhook design, failure handling

MODULE 10: DATABASE DESIGN & DATA MANAGEMENT

Every concept grounded in a real product decision

Foundations

- SQL vs NoSQL – a decision framework, not a features comparison
- When to use relational, document, wide-column, time-series, graph
- Real examples: Instagram uses PostgreSQL for profiles, Cassandra for feeds; LinkedIn uses graph DB for connections
- Data modeling, normalization, denormalization – when each applies

Performance

- Indexing – when it helps and when it becomes a liability
- Read-heavy vs write-heavy – Twitter (read) vs Uber (write) make different choices
- ACID properties and isolation levels – what you are buying with each guarantee

Scaling Data

- Partitioning and sharding – how it distributes data and what new problems it creates
- Consistent hashing – the idea, not the algorithm
- Hotspot problem – how Discord's popular server became a sharding problem
- CAP theorem – what it actually says, strong vs eventual consistency
- Real example: why Amazon cart shows stale data sometimes, why bank balance cannot

Caching

- Cache aside, write through, write back – when each pattern applies
- Cache invalidation – why it is hard, common approaches
- How Facebook uses Memcached, how Netflix caches at the edge
- Redis – sessions, leaderboards, pub/sub, distributed locks, rate limiting

Case Studies

- URL Shortener – ID generation, redirect performance, analytics
- Pastebin – blob vs relational, TTL expiry, abuse prevention

MODULE 11: DISTRIBUTED SYSTEMS & MESSAGING

How large systems communicate reliably at scale

Messaging Fundamentals

- Why async processing exists – decoupling, durability, load smoothing
- Queue vs pub/sub – order processing (queue) vs audit logging (pub/sub)
- Delivery guarantees – at most once, at least once, exactly once
- Dead letter queues – why every async system needs one
- Real example: how Swiggy decouples order placement from restaurant notification

Kafka

- What Kafka solves that a queue does not – retention, replay, fan-out
- Partitions and consumer groups – ordering within a partition
- Kafka as an event log – replaying events to rebuild state
- Real examples: LinkedIn built Kafka for activity feed fan-out; Uber processes millions of location events per second; Zomato fans out order status to payment, logistics, notification simultaneously

Event Driven Architecture

- Choreography vs orchestration – the architectural difference
- Saga pattern – distributed transactions in e-commerce
- Outbox pattern – reliable event publishing without dual-write problem

Reliability

- Service discovery – how services find each other dynamically
- Circuit breaker – how Netflix Hystrix prevented one slow service from cascading into full outage
- Distributed transactions – the coordination problem

Case Studies

- Uber – real time location ingestion, matching, trip state
- Swiggy / Zomato – order flow, dispatch, real time tracking

MODULE 12: MICROSERVICES & RELIABILITY ENGINEERING

When to split, how to operate, how to survive failures

Architecture

- Monolith vs microservices – when decomposition is worth the cost
- Service boundaries – draw around business domains, not technical layers
- Synchronous vs asynchronous inter-service communication tradeoffs
- Real examples: Amazon decomposed its monolith; Shopify runs a Rails monolith at scale and that is a valid choice

Architecture

- Retry and exponential backoff – why naive retries make outages worse
- Idempotency keys – making retries safe for payments and state changes
- Timeout strategies – not setting a timeout is itself a design decision
- Bulkhead pattern – how the AWS S3 2017 outage cascaded because services had no bulkhead
- Health checks – what they test and what they miss

Observability

- Logging, metrics, distributed tracing – what each tells you
- Why a single trace ID across services changes how you debug
- Real example: one slow DB query caused cascade across 12 microservices – tracing found it in minutes

Scale

- Multi-region systems – the consistency cost of geographic distance
- High availability – leader-follower replication, failover, split brain
- Read-your-own-writes – why posting a tweet and not seeing it is a consistency failure

MODULE 13: REAL-TIME SYSTEM DESIGN

Systems where latency is measured in milliseconds

Real-Time Communication

- WebSocket architecture at scale – millions of persistent connections
- Pub/sub for real-time delivery – fan-out strategies at different scales
- Offline users – queuing messages and syncing on reconnect
- Real example: how Slack maintains WebSocket connections across millions of concurrent users

Geo-Spatial Systems

- How location data is stored and queried at scale
- Geohash and quadtree – the concept behind proximity search

- Real time location updates – the write volume problem
- Real example: how Ola processes millions of driver pings per minute and matches to riders within seconds

Notifications and Rate Limiting

- Fan-out problem – what happens when a celebrity with 10M followers posts
- How LinkedIn handles connection notification fan-out
- How Twitter moved from push to pull for high-follower accounts
- Distributed rate limiter – coordination problem across multiple gateway nodes

Case Studies

- WhatsApp – message delivery, presence service, group chat fan-out
- Slack – WebSocket management, workspace isolation, message sync
- Facebook Nearby Friends – location update frequency, privacy model, proximity query
- Hotstar Live Streaming – 25 million concurrent viewers, ingest pipeline, CDN strategy

MODULE 14: LARGE SCALE SYSTEM DESIGN

The systems that appear most in senior engineering interviews

Each design covers: requirements, capacity estimation, component breakdown, deep dives, and tradeoff discussion.

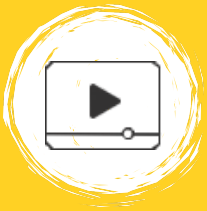
System Design Case Studies

- YouTube – upload pipeline, transcoding, CDN delivery, view count consistency
- Payment Gateway – ledger design, idempotency, saga pattern, reconciliation, what happens when UPI times out
- Search System – inverted index concept, indexing pipeline, ranking at architecture level
- BookMyShow – seat locking, flash sale traffic, virtual waiting room, concurrency control
- Stock Trading Platform – low latency design, order book, market data fan-out, why eventual consistency is unacceptable
- Distributed Cache – cluster architecture, eviction, consistency between cache and DB
- Recommendation System – architecture level, not the algorithm – candidate generation, ranking, serving layer
- Multi-Tenant SaaS – tenant isolation, shared vs dedicated infrastructure, data segregation

Enterprise Design Reviews

- Scalability analysis – identifying bottlenecks before they happen
- High availability design – what failure modes exist and how to handle each
- Failure handling – what breaks first, second, third under load

DELIVERABLES



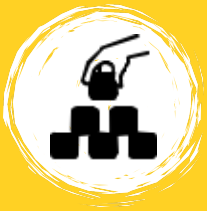
9.ON-DEMAND VIDEO COURSE CONTENT



10.DOWNLOADABLE RESOURCES



1.FROM SCRATCH TO MASTER LEVEL TRAINING



8.PLACEMENT UPDATES



2.RESUME BUILDING



7.Q&A SESSIONS



3.DAILY ASSIGNMENTS



6.COURSE COMPLETION CERTIFICATE



5.INTERVIEW GUIDANCE



4.LINKEDIN PROFILE BUILDING

TOP CITIES FOR SYSTEM DESIGN ENGINEERING DEMAND

- BANGALORE (BENGALURU)
- GURGAON (GURUGRAM)
- HYDERABAD
- PUNE
- MUMBAI
- CHENNAI
- NOIDA

KEY SECTORS DRIVING DEMAND FOR SYSTEM DESIGN ENGINEERING

- TECHNOLOGY AND SOFTWARE DEVELOPMENT
- E-COMMERCE HEALTHCARE
- FINANCE AND FINTECH
- HEALTHCARE
- EDUCATION AND EDTECH
- DATA SCIENCE AND ANALYTICS
- ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
- MEDIA AND ENTERTAINMENT
- STARTUPS AND TECH INCUBATORS

TOP COMPANIES HIRING



















ALL THE BEST

